

Implement a Man-in-the-middle Attacks to Inject False Data that Bypass DC State Estimation

1 Update measurements based on DC state estimation

You will implement this project on top of the communication networks and the power systems, i.e., the IEEE 9-bus system, in Project 2. In Table 1 of Project 2, we have listed the measurements for the IEEE 9-bus system. Note that those measurements are obtained through AC state estimation. In this project, we need to update measurements based on the DC state estimation. In Table 1, we have made the following changes to the measurements:

- (i) The voltage magnitude of all buses is changed to 1, and the reactive power injection of all buses is changed to 0. These two types of measurements will not be used in DC state estimation, but you should keep them in the communications implemented in this project.
- (ii) Active power injection of all buses is normalized on the base of 100 MW. Consequently, this makes the measurement vector as $\mathbf{z} = (0.67, 1.63, 0.85, 0.00, -0.90, 0.00, -1.00, 0.00, -1.25)^T$.
- (iii) *Note that in the DC state estimation, system state x is the voltage phasor for each bus.* Consequently, in Table 1, the voltage phasor for each bus (except the slack bus, whose voltage phasor is 0) is replaced by the variable name. This makes the state vector as $\mathbf{x} = (x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8)^T$.

Table 1. Measurements collected by communication nodes.

Substation Index	Measurements			Communication Node
	Type	Index	Values	
Bus 1	Voltage magnitude	11	1.00	Relay 1
	Voltage phasor	12	0.00	
	Real power injection	13	0.67	
	Reactive power injection	14	0.00	
Bus 2	Voltage magnitude	21	1.00	
	Voltage phasor	22	x_1	
	Real power injection	23	1.63	
	Reactive power injection	24	0.00	
Bus 3	Voltage magnitude	31	1.00	Relay 2
	Voltage phasor	32	x_2	
	Real power injection	33	0.85	
	Reactive power injection	34	0.00	
Bus 4	Voltage magnitude	41	1.00	
	Voltage phasor	42	x_3	
	Real power injection	43	0.00	
	Reactive power injection	44	0.00	
Bus 5	Voltage magnitude	51	1.00	Relay 3
	Voltage phasor	52	x_4	
	Real power injection	53	-0.90	

	Reactive power injection	54	0.00	
Bus 6	Voltage magnitude	61	1.00	
	Voltage phasor	62	x_5	
	Real power injection	63	0.00	
	Reactive power injection	64	0.00	Relay 4
Bus 7	Voltage magnitude	71	1.00	
	Voltage phasor	72	x_6	
	Real power injection	73	-1.00	
	Reactive power injection	74	0.00	
Bus 8	Voltage magnitude	81	1.00	
	Voltage phasor	82	x_7	
	Real power injection	83	0.00	
	Reactive power injection	84	0.00	
Bus 9	Voltage magnitude	91	1.00	
	Voltage phasor	92	x_8	
	Real power injection	93	-1.25	
	Reactive power injection	94	0.00	

For this power system, the H measurement matrix is:

$$\begin{bmatrix}
 0 & 0 & -17.3611 & 0 & 0 & 0 & 0 & 0 \\
 16.0000 & 0 & 0 & 0 & 0 & 0 & -16.000 & 0 \\
 0 & 17.0648 & 0 & 0 & -17.0648 & 0 & 0 & 0 \\
 0 & 0 & 39.9954 & -10.8696 & 0 & 0 & 0 & -11.7647 \\
 0 & 0 & -10.8696 & 16.7519 & -5.8824 & 0 & 0 & 0 \\
 0 & -17.0648 & 0 & -5.8824 & 32.8678 & -9.9206 & 0 & 0 \\
 0 & 0 & 0 & 0 & -9.9206 & 23.8095 & -13.8889 & 0 \\
 -16.0000 & 0 & 0 & 0 & 0 & -13.8889 & 36.1001 & -6.2112 \\
 0 & 0 & -11.7647 & 0 & 0 & 0 & -6.2112 & 17.9759
 \end{bmatrix}$$

And matrix W , which is a diagonal matrix whose elements are reciprocals of the variances of meter errors, is:

$$\begin{bmatrix}
 100 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 100 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 100 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 100 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 100 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 100 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 100 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 100 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 100
 \end{bmatrix}$$

2 Implementation

In this project, the assignment is to use the Data Aggregator machine to perform a man-in-the-middle attack, which applies false data injection attacks (FDIAs). Specifically, while the Data Aggregator receives measurements from four relays, it changes the measurements based on the results of false data injection.

Please download the starter code at <https://github.com/hugolin615/ELE420520Spring2026/tree/main/project3>.

Specifically, your implementations are marked by TODO in the following files:

- ***calculate_fdia.m***: using the formula in the lecture note to calculate the estimated values of the original system states. (In the codes, I have put the accuracy of results at the 2 digits after the decimal point). (***Hint: you should use Matlab or GNU Octave, which is an open-source Matlab alternative, to run this script***). In the Raspberry PI machine, you should be able to install Octave by running the command "sudo apt install octave."
- ***data_aggregator.py***: when the Data Aggregator machine combines measurements from multiple relays, you should replace the measurements and system states in the packed response that will be sent to the Control Center machine. You should input the appropriate values based on what is calculated from *calculate_fdia.m* in the *fdia_measure*.
- ***relay1.py*, *relay2.py*, *relay3.py*, and *relay4.py***: in these scripts, you only need to update the estimated state values in the *measurement* with the appropriate values obtained from *calculate_fdia.m*.

To validate your implementation, the false data injection attacks change the measurement from 1.63 to 1.79 indexed by 23, according to my reference solution. Consequently, when *data_aggregator.py* prints out measurements, measurement 23 collected from relay 1 should have the value of 1.63, but measurement 23, which the data aggregator issued to the control center, should have the value of 1.79.

3 Turn-in and Grading.

Please submit a **single PDF** file, named as "***Last Name_First Name_Project3.pdf***", including the following contents:

- **Python scripts (*calculate_fdia.m*, *data_aggregator.py*, *relay1.py*, *relay2.py*, *relay3.py*, and *relay4.py*) including your implementation (30%).**
- **A snapshot of running *calculate_fdia.m* in Matlab or GNU Octave (25%).** After running the scripts, take a **snapshot** to show the values of estimated states and measurements before and after FDIA. Note that the print statements are already provided in the startup code.
- **A snapshot of running the Python scripts in Mininet (15%).** After running the scripts, take a snapshot to show the **consoles** of all machines. I suggested putting machines in the pattern shown in Figure 1 so that I can compare the outputs from the Control Center machine and the Data Aggregator machine clearly.

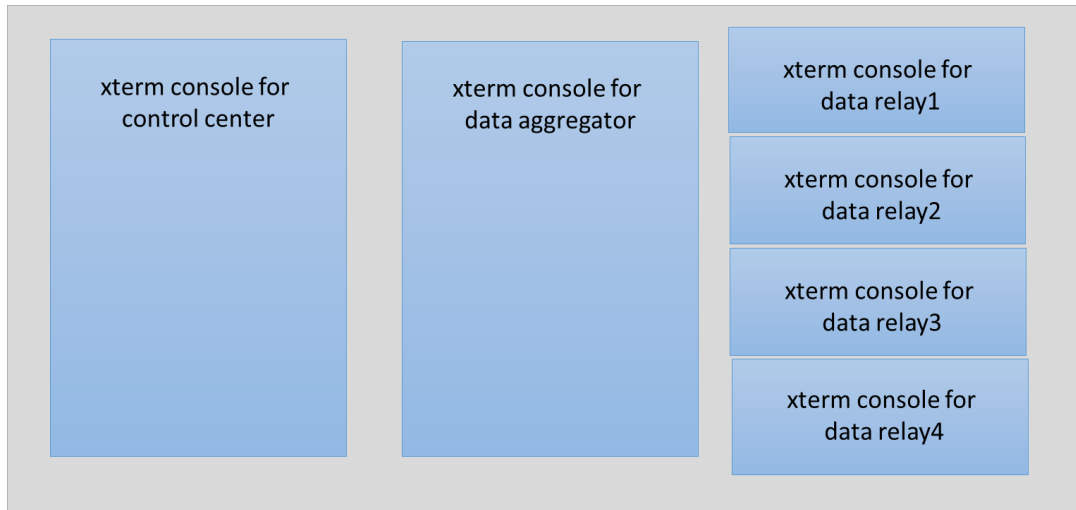


Figure 1. Suggested pattern to show all running machines in mininet.

Also submit a separate Network trace file recorded by Wireshark (30%), not included in the pdf file. In Mininet, use Xterm to open another console for the Data Aggregator machine and start Wireshark in that console. Using Wireshark to monitor the network traffic that will go through the Data Aggregator machine, while you run your Python scripts. Save what you recorded as a network trace file (in the format of .pcapng or .pcap). Please name this trace file as *mitm.pcap* or *mitm.pcapng*.